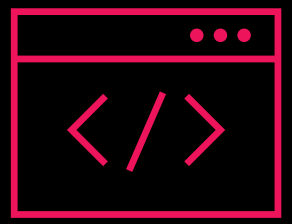
FIAP

GIT, GITHUB E PYTHON (VARIÁVEIS E OPERADORES)

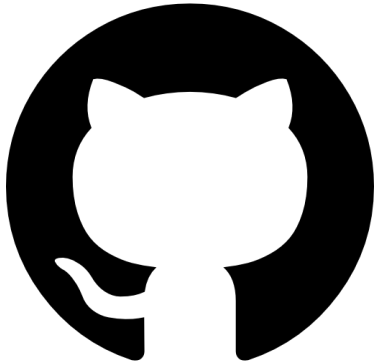
Pensamento Computacional e Automação com Python



Alexandre Russi Jr.

profalexandre.junior@fiap.com.br

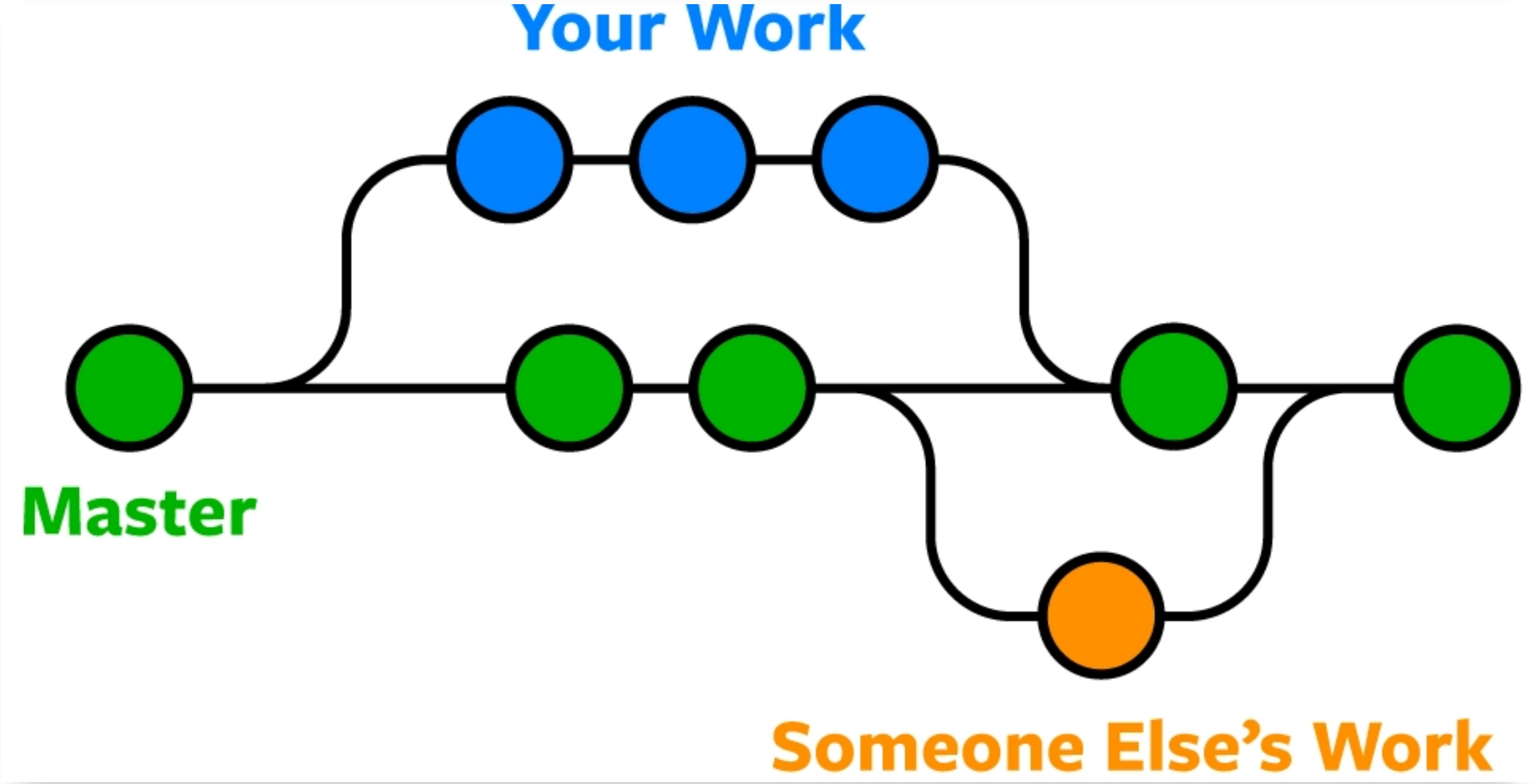
GIT vs. GITHUB



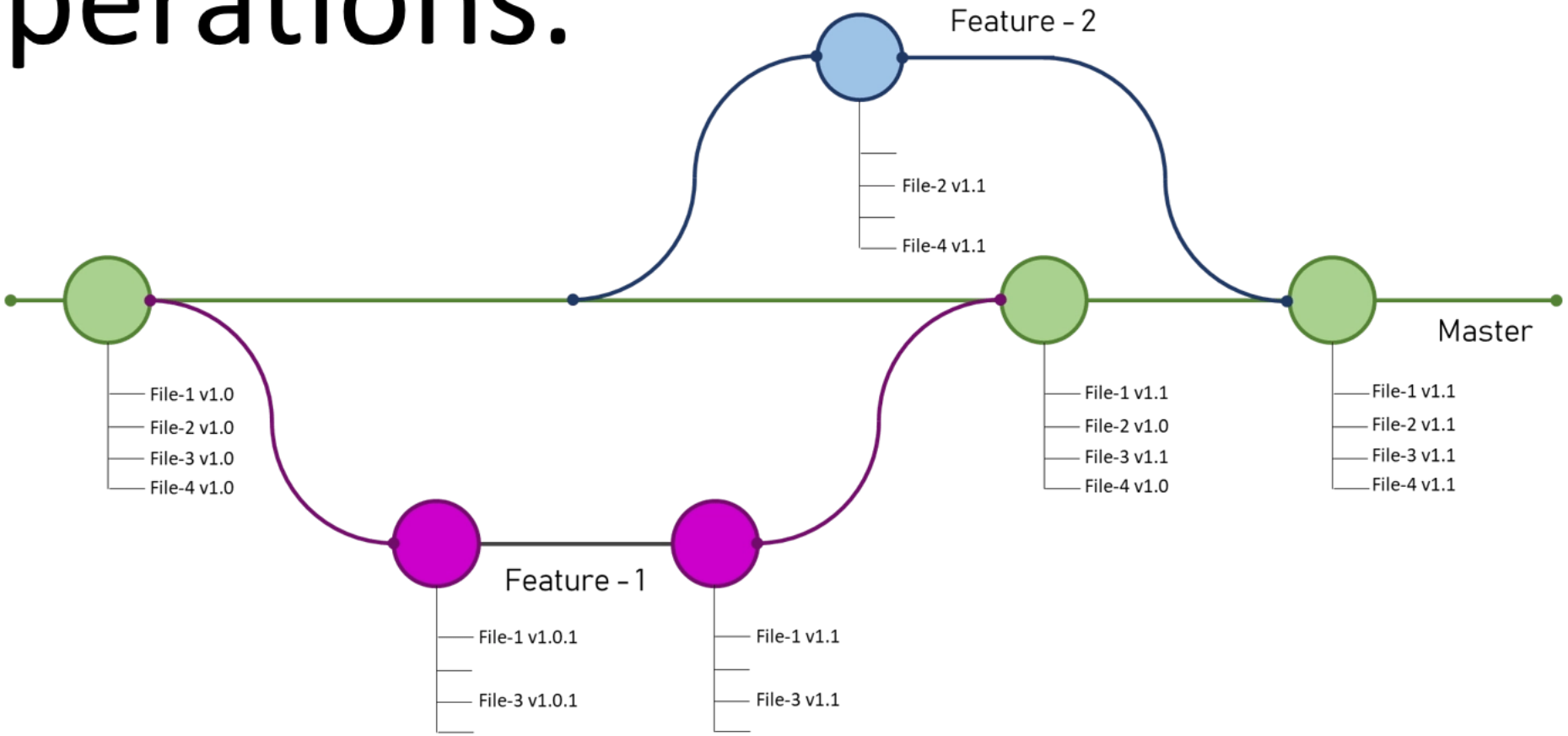
Git: versionador de códigos

Registra histórico

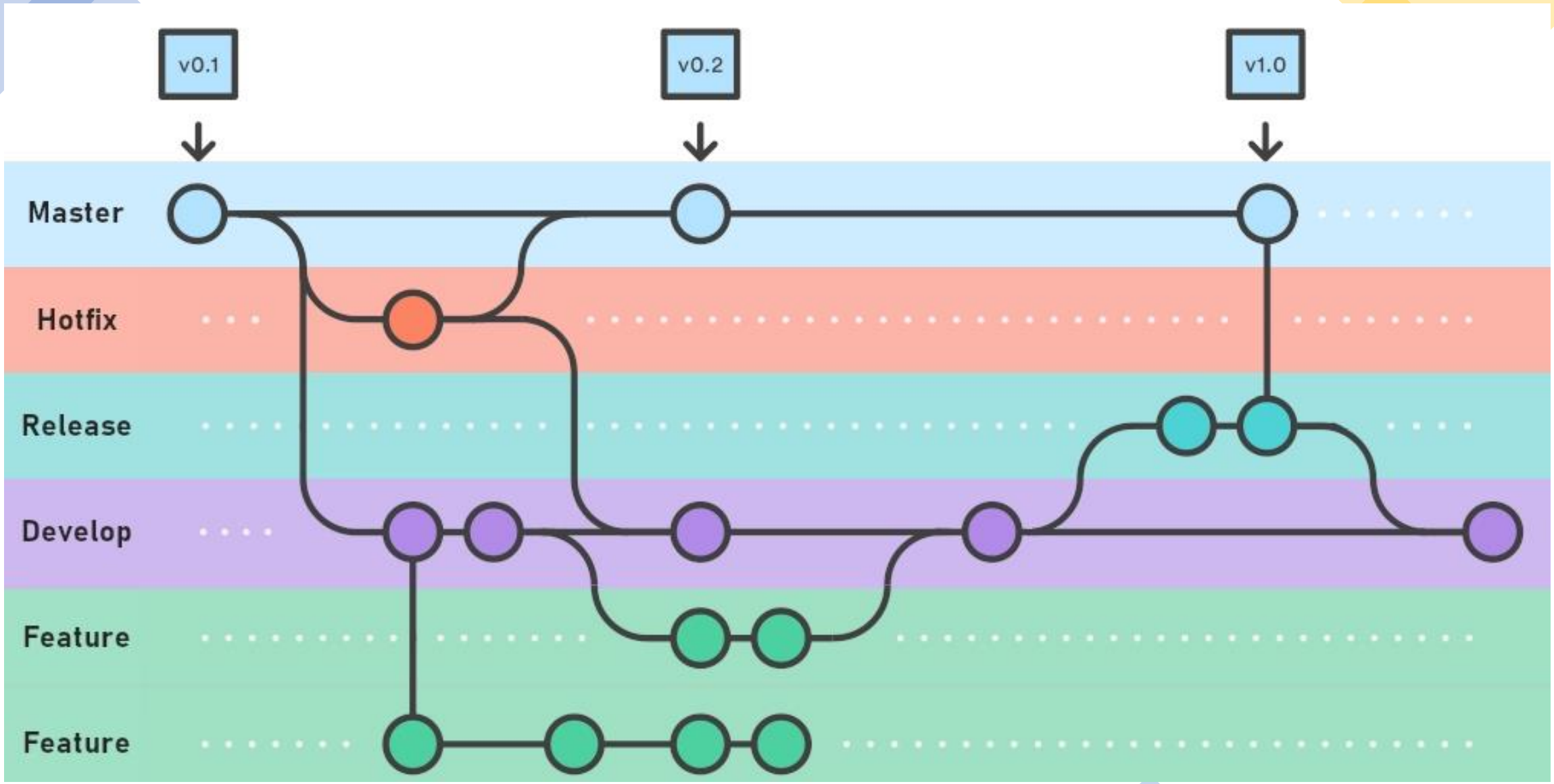
GitHub: plataforma de hospedagem de código-fonte e arquivos com controle de versão do Git

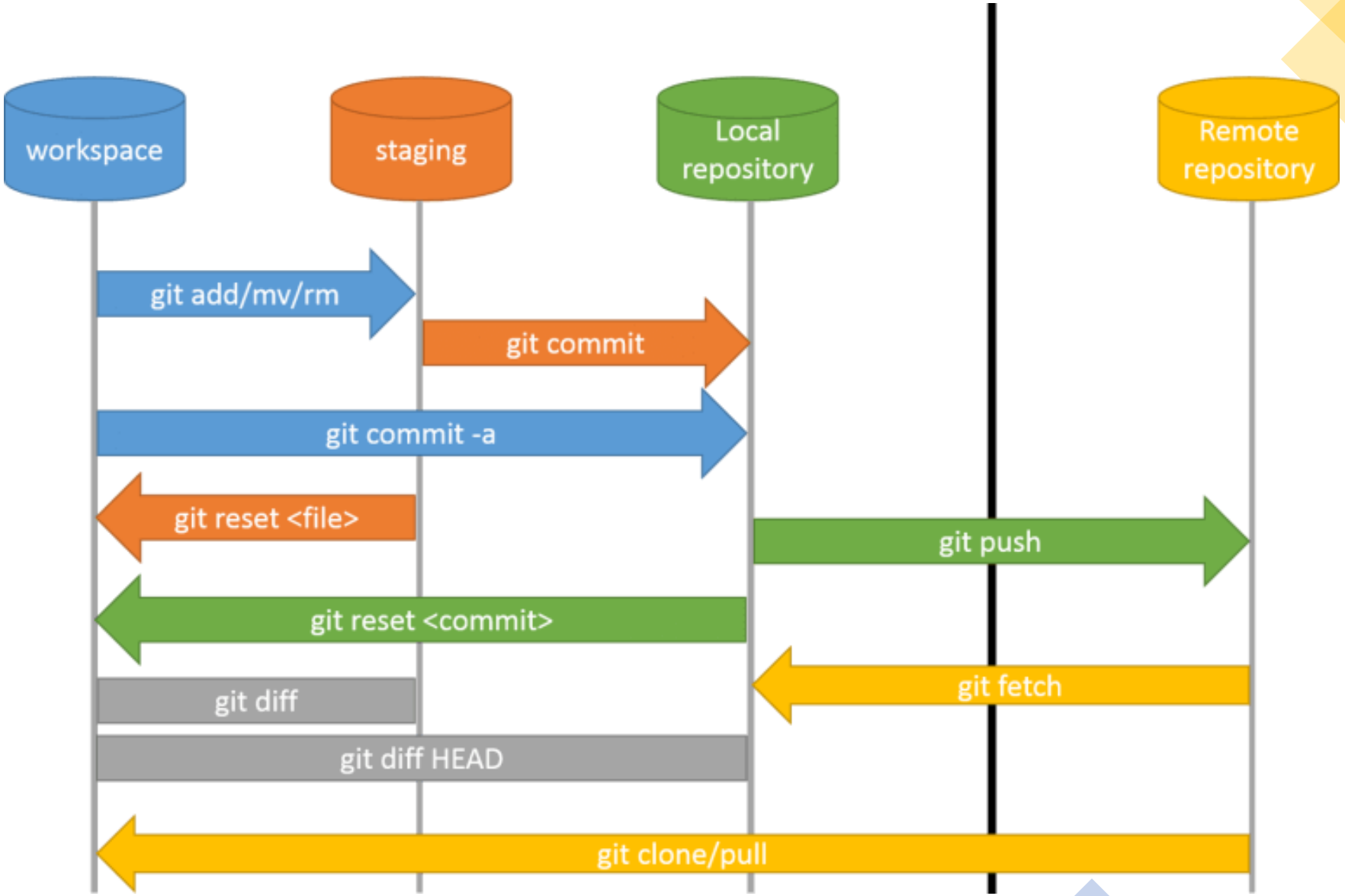

Your Work
Master
Someone Else's Work

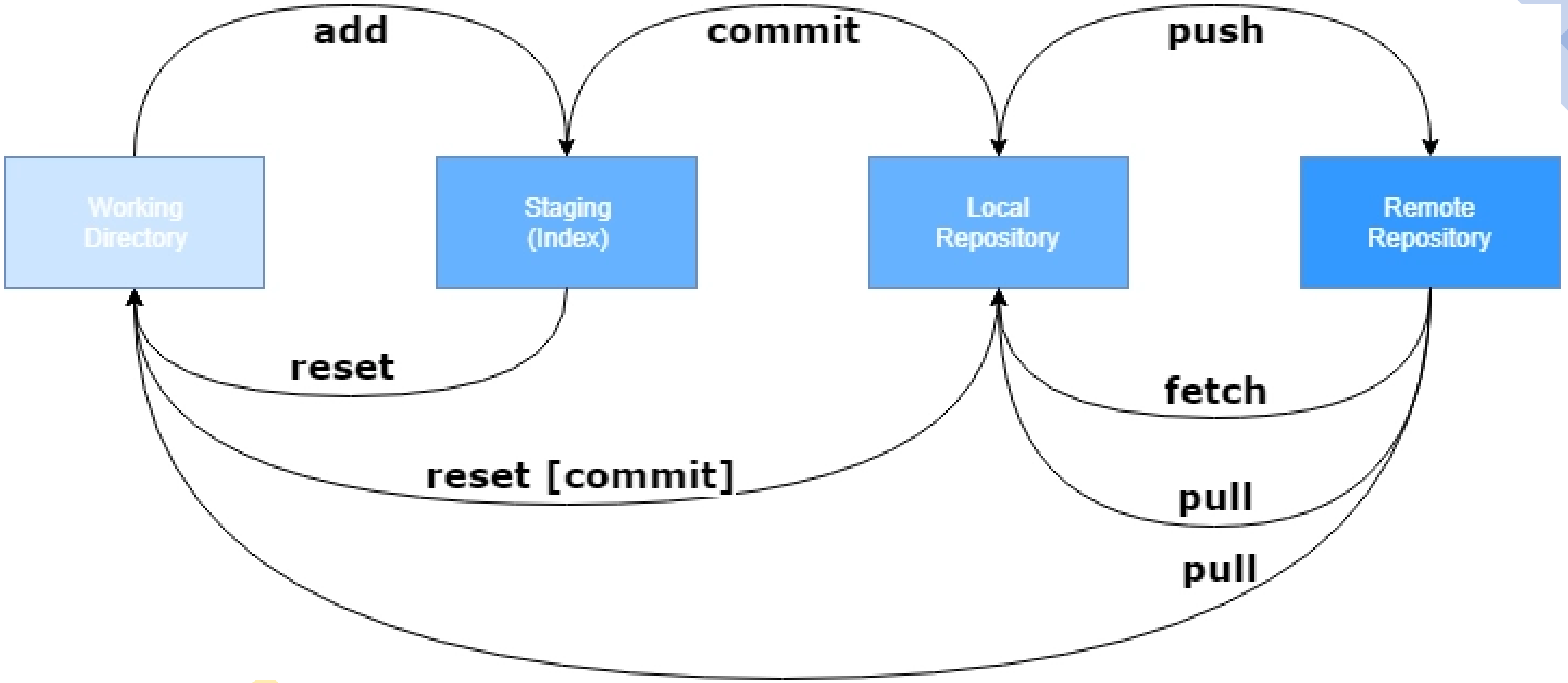
GIT Branch and its Operations.



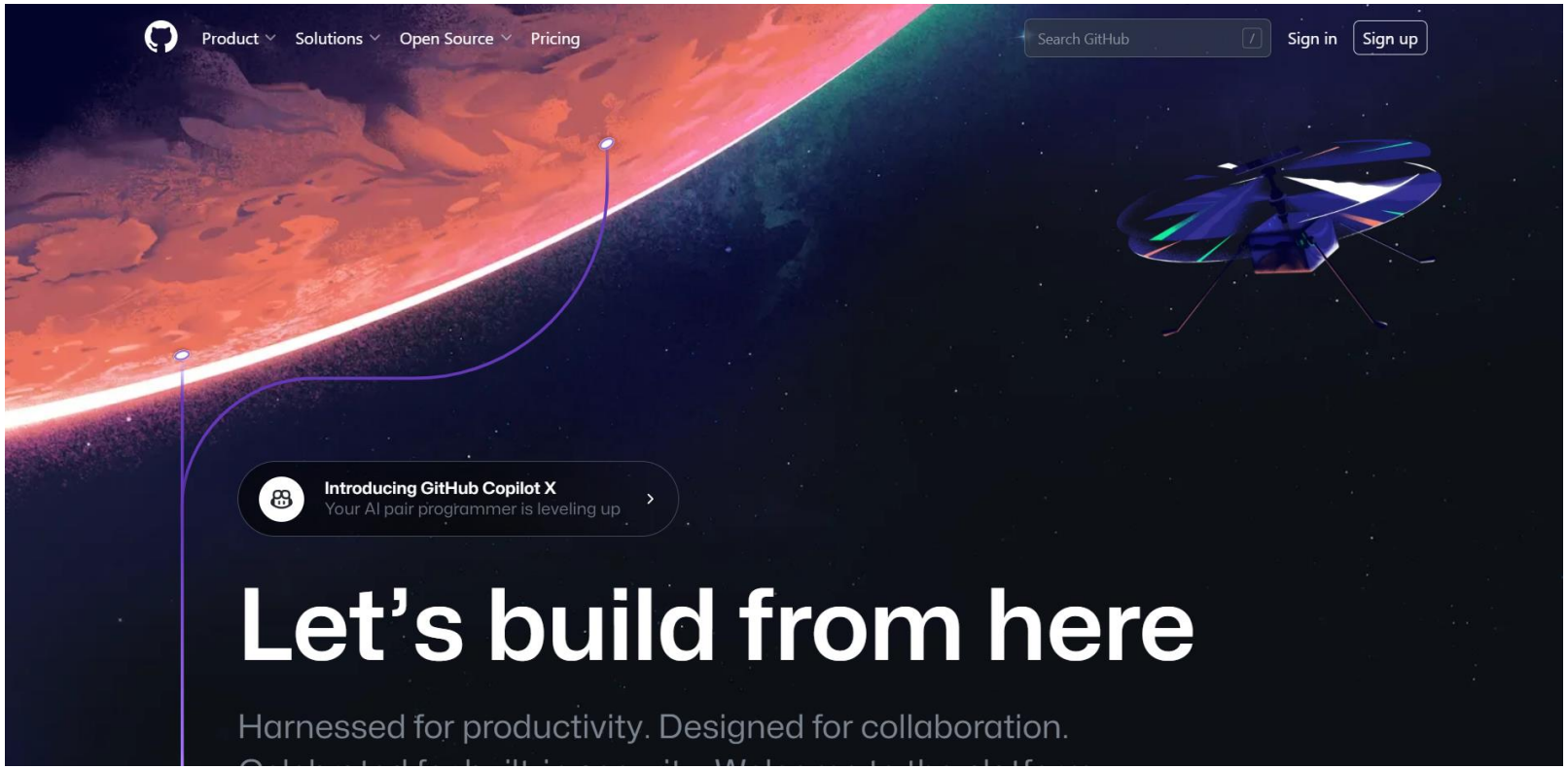
MASTER
DEVELOP
FEATURE
FEATURE


v0.1
v0.2
v1.0
Master
Hotfix
Release
Develop
Feature
Feature


workspace
staging
Local repository
Remote repository
git add/mv/rm
git commit
git commit -a
git reset <file>
git push
git reset <commit>
git fetch
git diff
git diff HEAD
git clone/pull

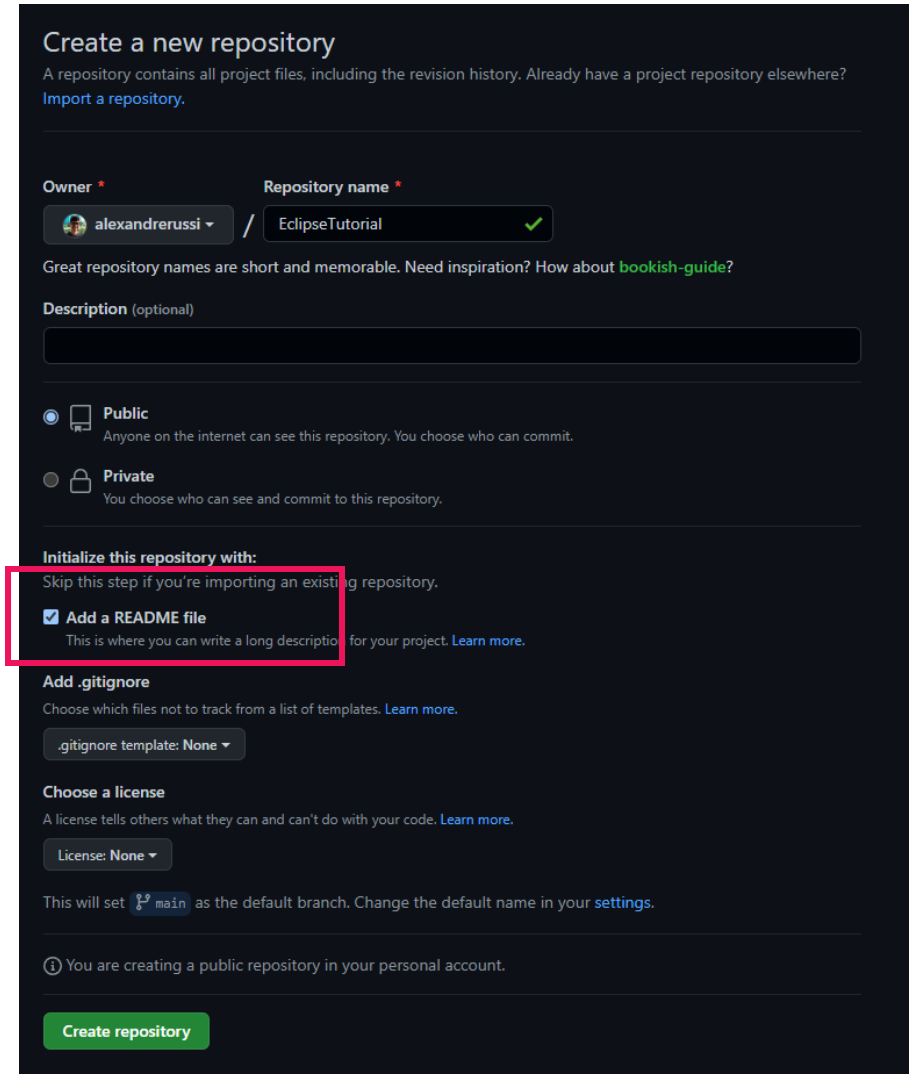

add
commit
push
Working Directory
Staging (Index)
Local Repository
Remote Repository
reset
fetch
reset [commit]
pull
pull

CRIANDO CONTA NO GITHUB



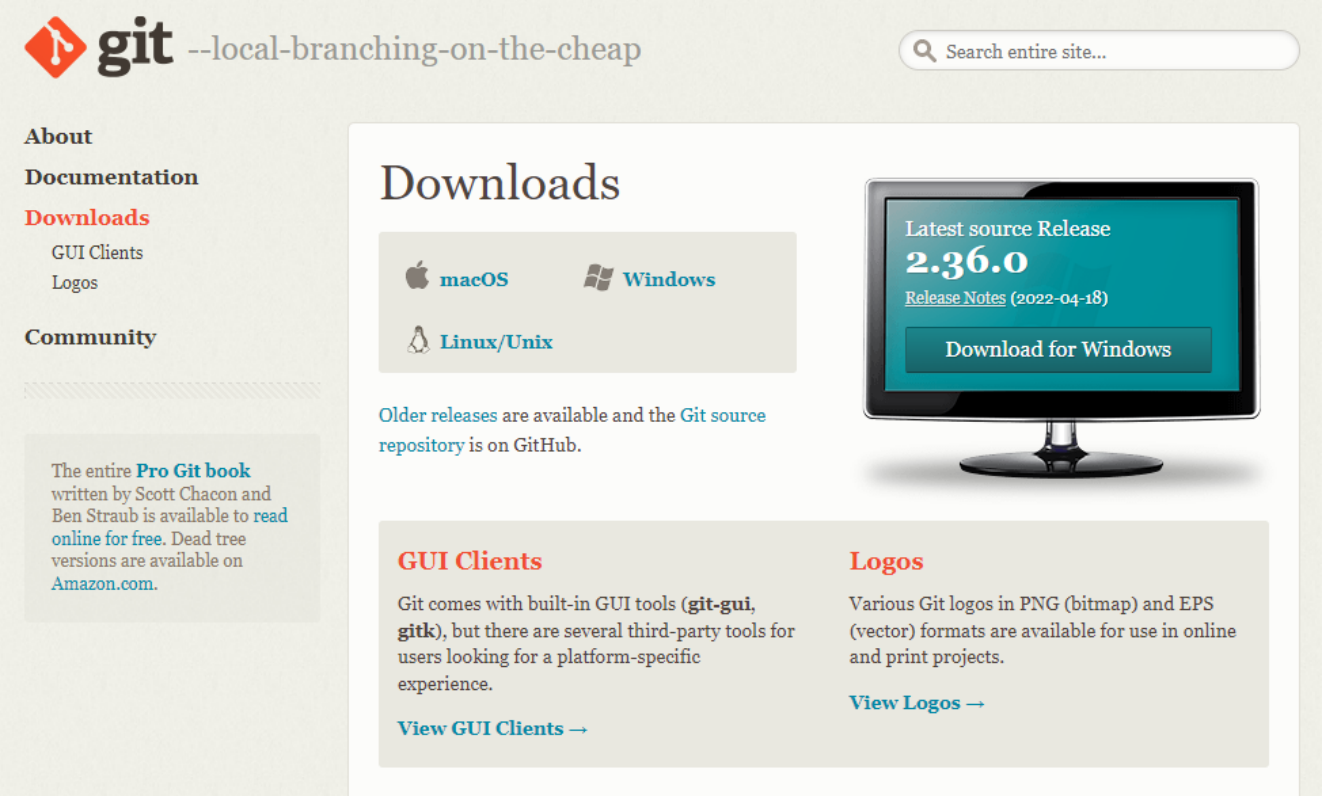
https://github.com/

CRIAR NOVO REPOSITÓRIO NO GITHUB



DOWNLOAD GIT

- https://git-scm.com/downloads



CONFIGURE SEU GIT COM EMAIL E USERNAME DO GITHUB

- Abra o git bash
- Insira os comandos:
  - git config --global user.name “username”
  - git config --global user.email “email@email.com”

```
MINGW64:/c/Users/junio

junio@PC-Russi MINGW64 ~
$ git config --global user.name "alexandrerussi"

junio@PC-Russi MINGW64 ~
$ git config --global user.email "alexandrerussijunior@gmail.com"

junio@PC-Russi MINGW64 ~
$
```

HISTÓRICO

- Python é uma linguagem de programação de alto nível, criada por Guido van Rossum e lançada pela primeira vez em 1991. Guido desenvolveu Python enquanto trabalhava no CWI (Centrum Wiskunde & Informatica) na Holanda. Ele queria criar uma linguagem que fosse fácil de ler, com uma sintaxe limpa e simples, inspirada em parte pela linguagem ABC, da qual ele era um dos desenvolvedores.

- O nome "Python" foi escolhido não pela serpente, mas em homenagem ao programa de televisão britânico "Monty Python's Flying Circus", do qual Guido era fã. Essa escolha reflete a abordagem lúdica e irreverente que ele queria para a linguagem.

HISTÓRIA

- 1995: recebeu financiamento da DARPA para CP4E
- 2001: Python Software Foundation – comunidade
  - Patrocinada pelo Google, Microsoft, globo.com etc..
- Linguagem padrão em alguns OS's (MacOS, Linux etc)
- RaspberryPi – Projetos IoT

PYTHON É...

- Linguagem de propósito geral
- Fácil e intuitivo
- Multiplataforma
  - Irá funcionar em muitas plataformas
- "Batteries Included"
  - Vem com a maioria das coisas
- Código aberto – livre
- Organizada
- Orientada a objetos

ZEN DO PYTHON

- Bonito é melhor que feio.
- Explícito é melhor que implícito.
- Simples é melhor que complexo.
- Complexo é melhor que complicado.
- Linear é melhor do que aninhado.
- Esparso é melhor que denso.
- Legibilidade conta.
- Casos especiais não são especiais o bastante para quebrar as regras.
- Ainda que praticidade vença a pureza.
- Erros nunca devem passar silenciosamente.
- A menos que sejam explicitamente silenciados.
- Diante da ambiguidade, recuse a tentação de adivinhar.
- Dever haver um — e preferencialmente apenas um — modo óbvio para fazer algo.
- Embora esse modo possa não ser óbvio a princípio a menos que você seja holandês.
- Agora é melhor que nunca.
- Apesar de que nunca normalmente é melhor do que *exatamente* agora
- Se a implementação é difícil de explicar, é uma má ideia
- Se a implementação é fácil de explicar, pode ser uma boa ideia
- Namespaces são uma grande ideia — vamos ter mais dessas!

COMO FUNCIONA O PYTHON

- Linguagens de alto nível: são linguagens mais próximas da linguagem humana, com uma sintaxe mais fácil de entender e escrever. Elas são projetadas para serem mais acessíveis e amigáveis aos programadores. Exemplos de linguagens de alto nível incluem Python, Java, C++, e JavaScript.

- Linguagens de baixo nível: são linguagens mais próximas da linguagem de máquina e do hardware do computador. Elas são mais difíceis de ler e escrever, pois estão mais próximas da linguagem binária que os computadores entendem diretamente. Exemplos de linguagens de baixo nível incluem Assembly e linguagens específicas de arquitetura, como MIPS ou x86.

COMO FUNCIONA O PYTHON

- Linguagem Compilada
  - **Como funciona**: O código-fonte é traduzido de uma só vez para código de máquina (executável) por um compilador.
  - **Exemplo de linguagens**: C, C++, Go, Rust.
  - **Vantagens:**
    - Execução geralmente mais rápida, porque já está tudo traduzido.
    - Pode ser otimizado pelo compilador.
  - **Desvantagens:**
    - Processo de compilação pode ser mais demorado.
    - Mais difícil de depurar (debugar), pois o erro aparece depois da compilação.
- **Resumo:** Você escreve o código → compila → executa.

- Linguagem Interpretada
  - **Como funciona**: O código-fonte é executado linha por linha por um interpretador.
  - **Exemplo de linguagens**: Python, JavaScript, Ruby.
  - **Vantagens:**
    - Mais fácil de testar e depurar, pois os erros aparecem na hora.
    - Mais flexível para scripts e prototipação.
  - **Desvantagens:**
    - Mais lenta na execução, pois interpreta o código em tempo real.
- **Resumo:** Você escreve o código → interpreta e executa direto.

COMO FUNCIONA O PYTHON

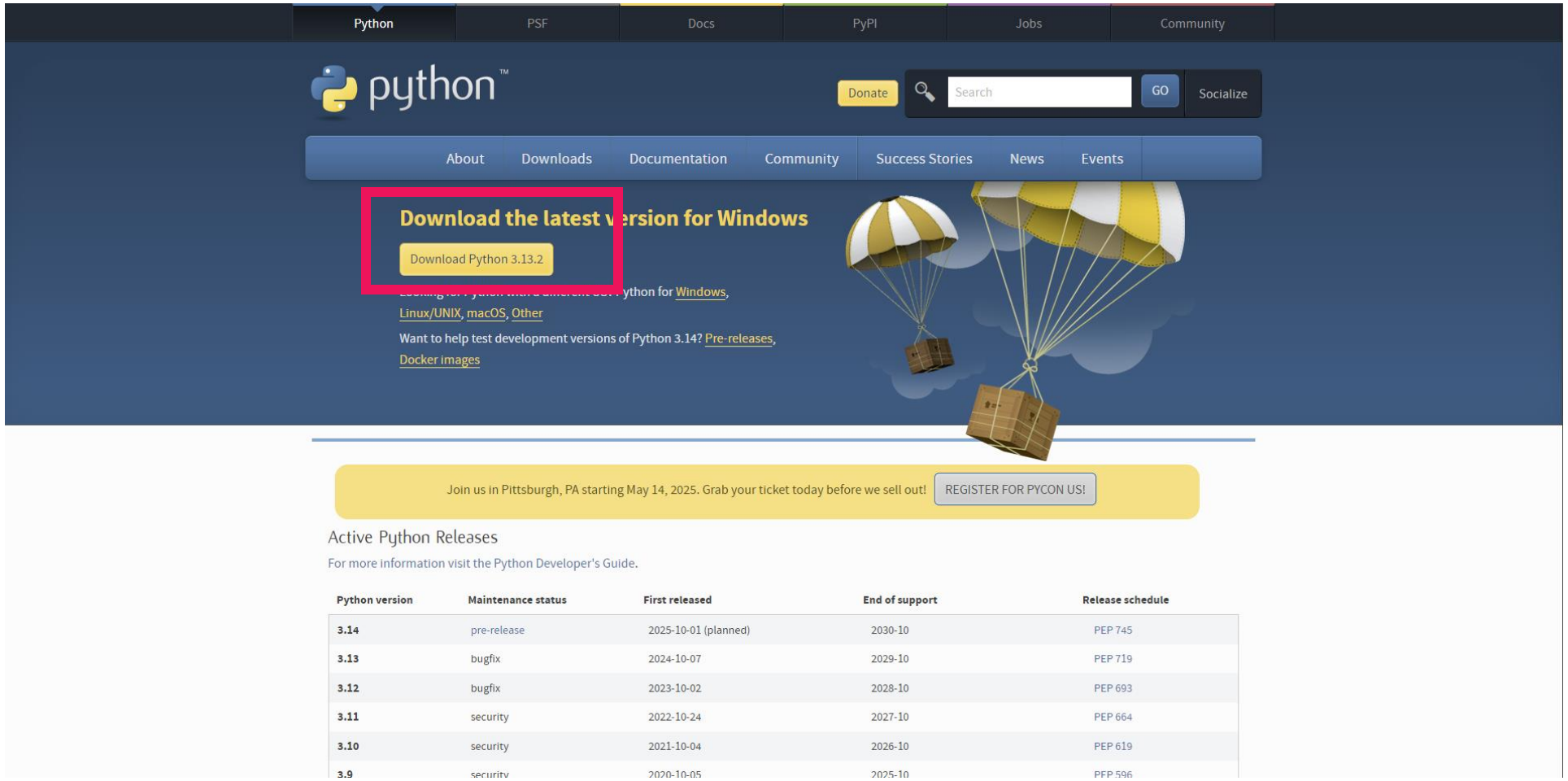
- Precisamos instalar o intepretador python no Windows

```
C:\WINDOWS\system32\cmd.

C:\Users\alexa>python --version
Python nÒo foi encontrado; executar sem argumentos para instalar do Microsoft Store ou desabilitar este atalho em Configuraþ§es > Aplicativos > Configuraþ§e
s avanþadas do aplicativo > Aliases de execuþÒo do aplicativo.

C:\Users\alexa>
```

INSTALAÇÃO

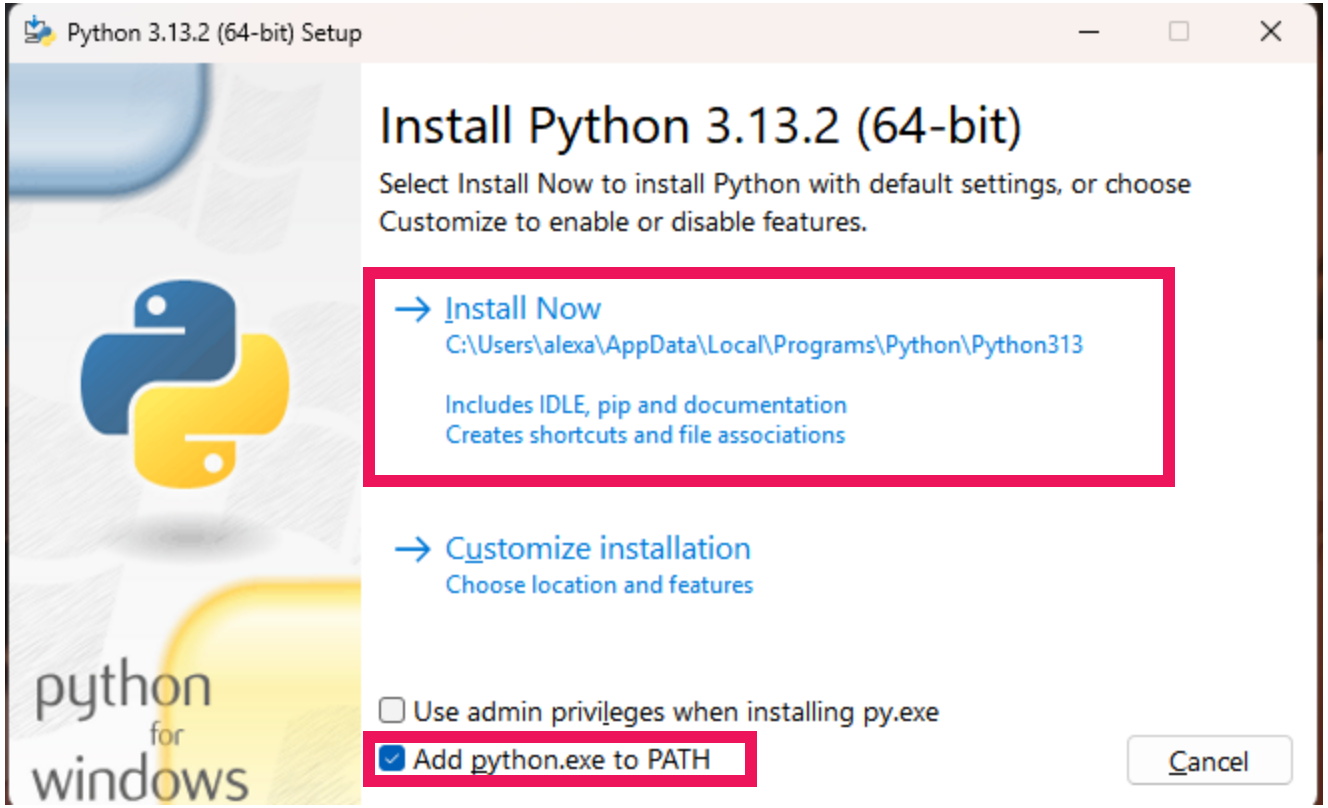


https://www.python.org/downloads/

INSTALE O PYTHON

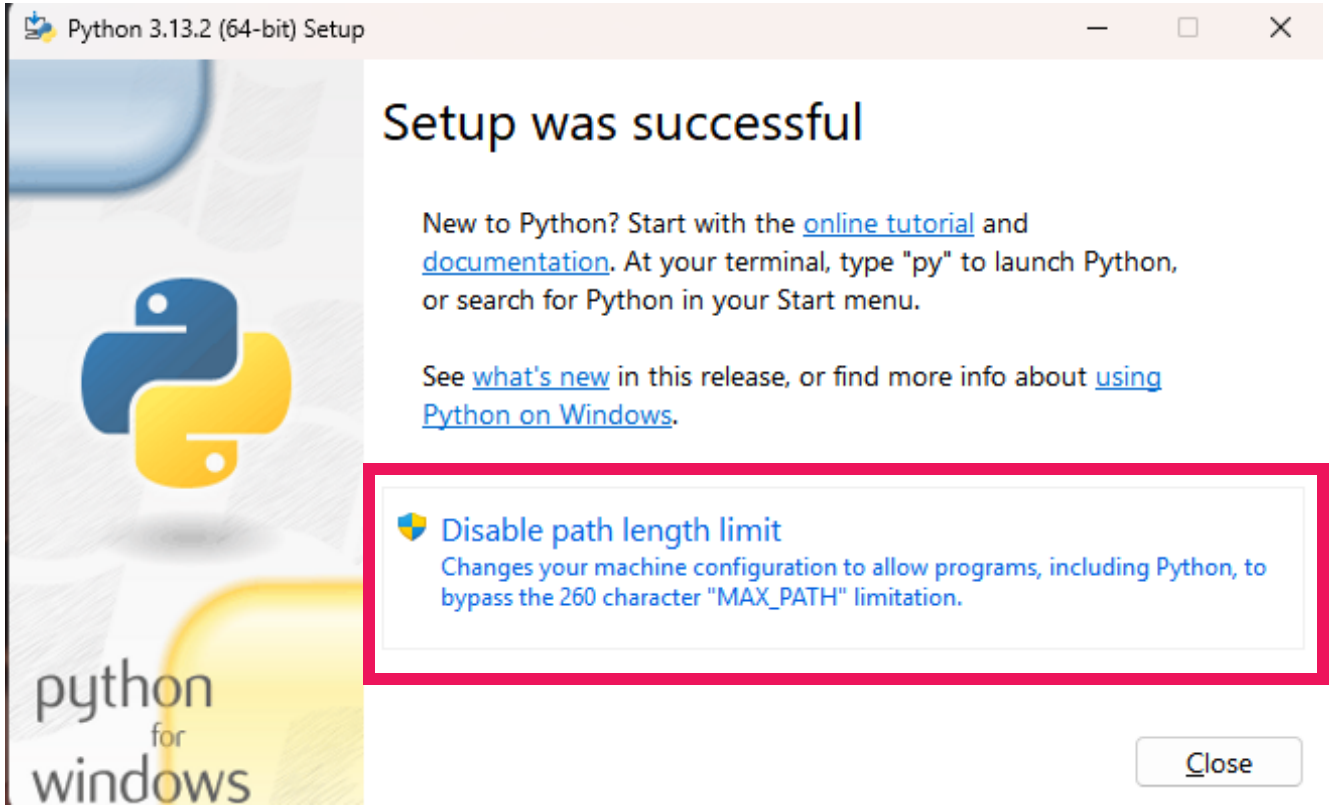
Selecione a opção marcada “Add python.exe to PATH” para que o comando python seja reconhecida no seu PC e CMD

A seguir, clique em Install Now



DESABILITE O LIMITE DE CARACTERES

Clique para desabilitar esse limite de caracteres.



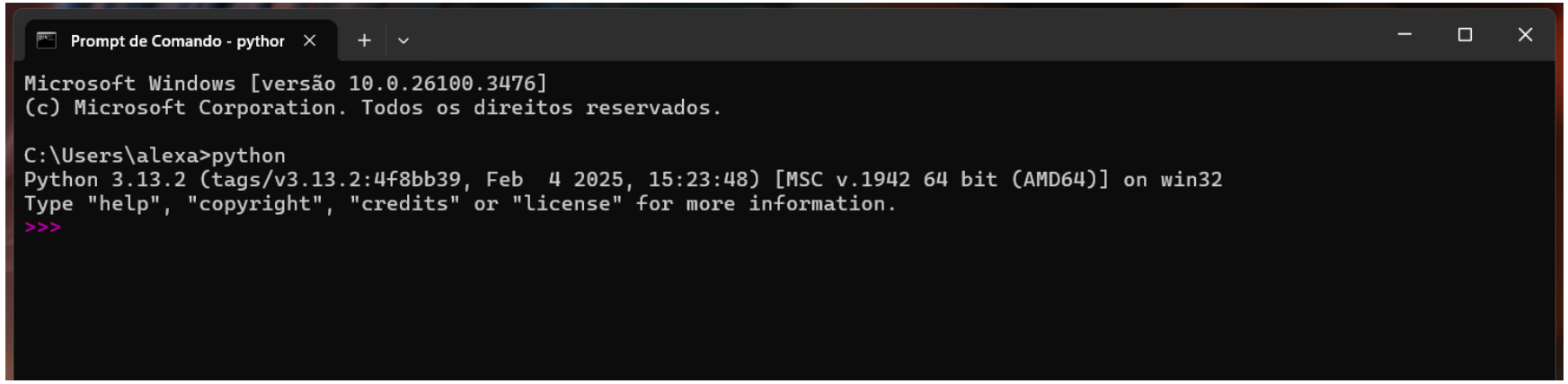
TESTE O PYTHON

Feche qualquer prompt de comando que você tenha ativo.

Abra um novo CMD e teste o comando “python”

Se abrir como mostra na figura, sucesso!

Se não, repita o processo.



ZEN DO PYTHON NO CMD

```
Prompt de Comando - pythor

Microsoft Windows [versão 10.0.26100.3476]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\alexa>python
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>>
```

IDEs

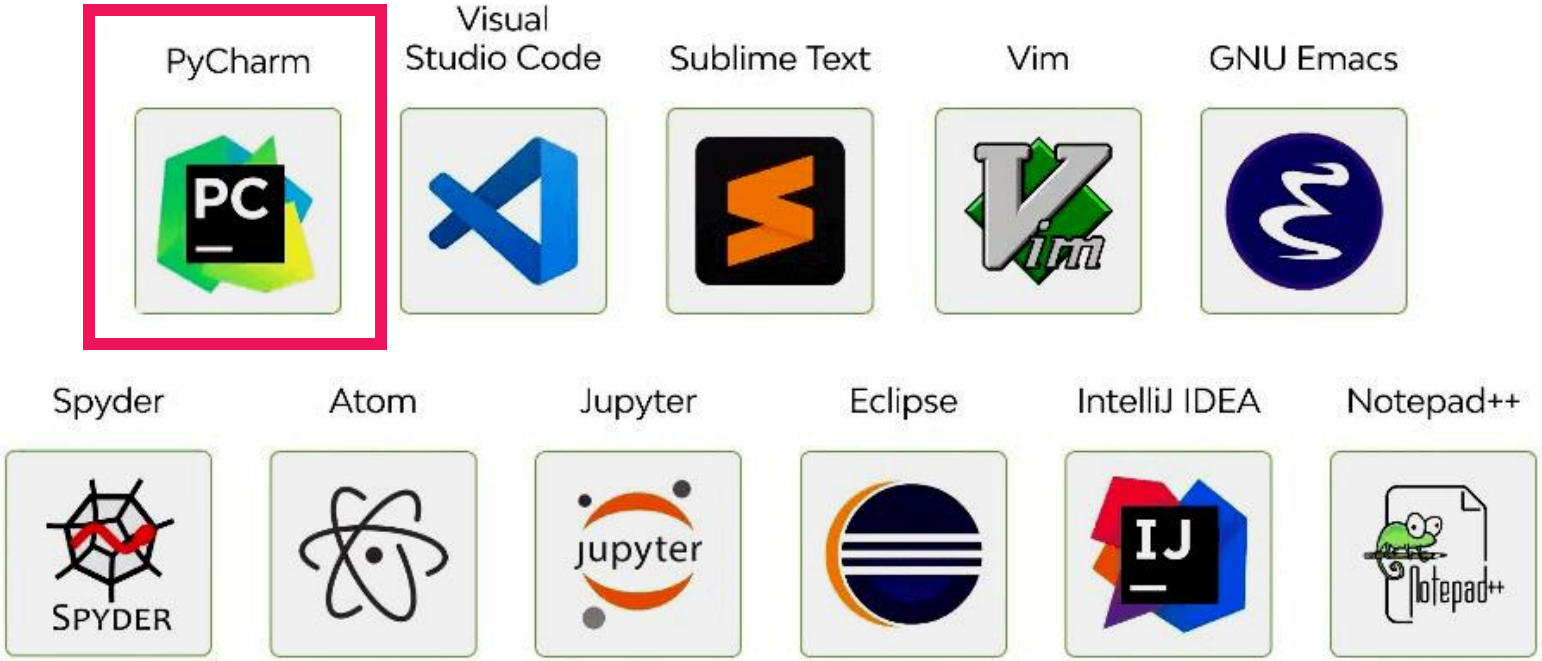
Uma **IDE** (**Integrated Development Environment** ou **Ambiente de Desenvolvimento Integrado**) é um programa que visa maximizar a **produtividade** do desenvolvedor, que tem várias funcionalidades que auxiliam no desenvolvimento, entre elas, as mais comuns são:

- **Editor**: editor de código-fonte específico para cada linguagem de programação suportada pela IDE.
- **Compilador**: compila o código-fonte.
- **Debugger (Depurador):** executa o programa "passo-a-passo", por meio do qual é possível verificar o que ocorre em cada linha do programa, auxiliando no entendimento do sistema e no processo de encontrar e corrigir problemas.

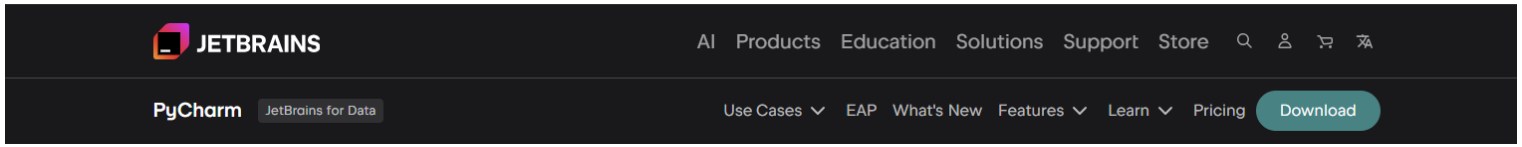
IDEs

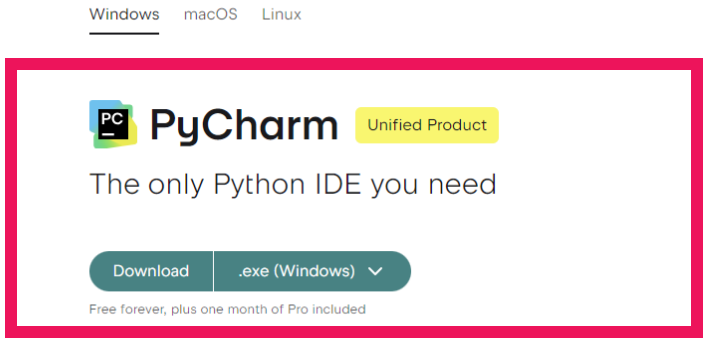
- **Modelagem**: criação de modelos de classes, objetos, interfaces, associações e interações de forma visual.
- **Geração de código**: geração de código a partir de templates de código comumente utilizados para solucionar problemas rotineiros.
- **Deploy (Distribuição):** auxilia no processo de gerar o arquivo final para a instalação do programa desenvolvido.
- **Testes automatizados**: realiza testes no programa de forma automatizada, baseados em scripts ou programas de testes previamente especificados, gerando relatórios que auxiliam na análise do impacto das alterações no código-fonte.
- **Refactoring (Refatoração)**: consiste na melhoria constante do código-fonte do programa

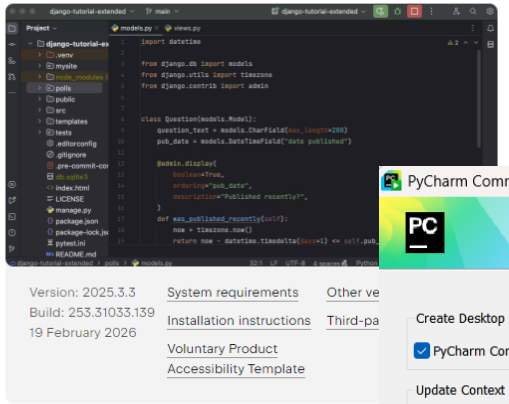
PRINCIPAIS IDEs e EDITORES DE CÓDIGO PARA PYTHON

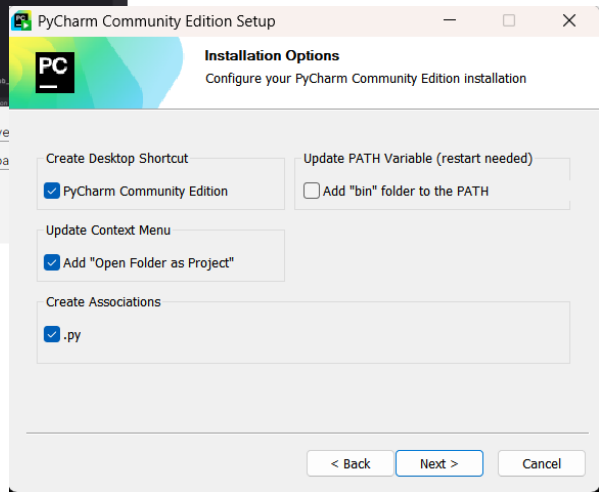


INSTALAÇÃO PYCHARM









PyCharm is now one unified product!

All users now automatically start with a free one-month Pro trial. After that, you can subscribe to Pro or keep using the core features for free – now with Jupyter support included.

https://www.jetbrains.com/pycharm/download/?section=windows

PRIMEIROS COMANDOS COM PYTHON

PRIMEIROS COMANDOS COM PYTHON

PRIMEIROS COMANDOS COM PYTHON

```
nome = input ('Qual é seu nome?')
idade = input ('Quantos anos você tem?')
peso = input ('Quanto você peso?')
print ( nome , idade , peso )
```

ATIVIDADE

FIAP

Desafio 01

Crie um script Python que leia o nome de uma pessoa e mostre uma mensagem de boas-vindas de acordo com o valor digitado.

Desafio 02

Crie um script Python que leia o dia, o mês e o ano de nascimento de uma pessoa e mostre uma mensagem com a data formatada.

Desafio 03

Crie um script Python que leia dois números e tente mostrar a soma entre eles.

10 minutos!!

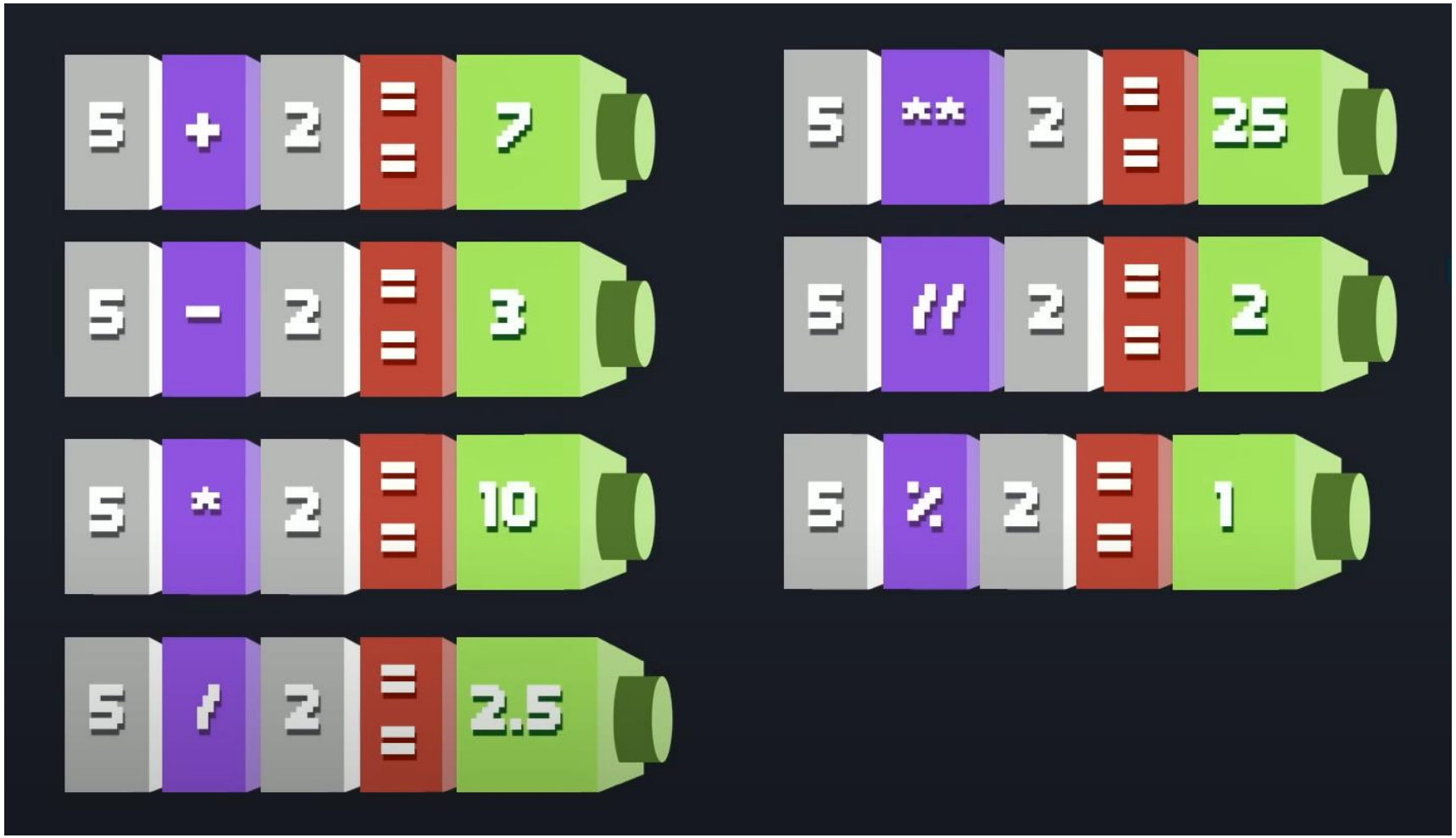
TIPOS PRIMITIVOS

TIPOS PRIMITIVOS DE VARIÁVEIS

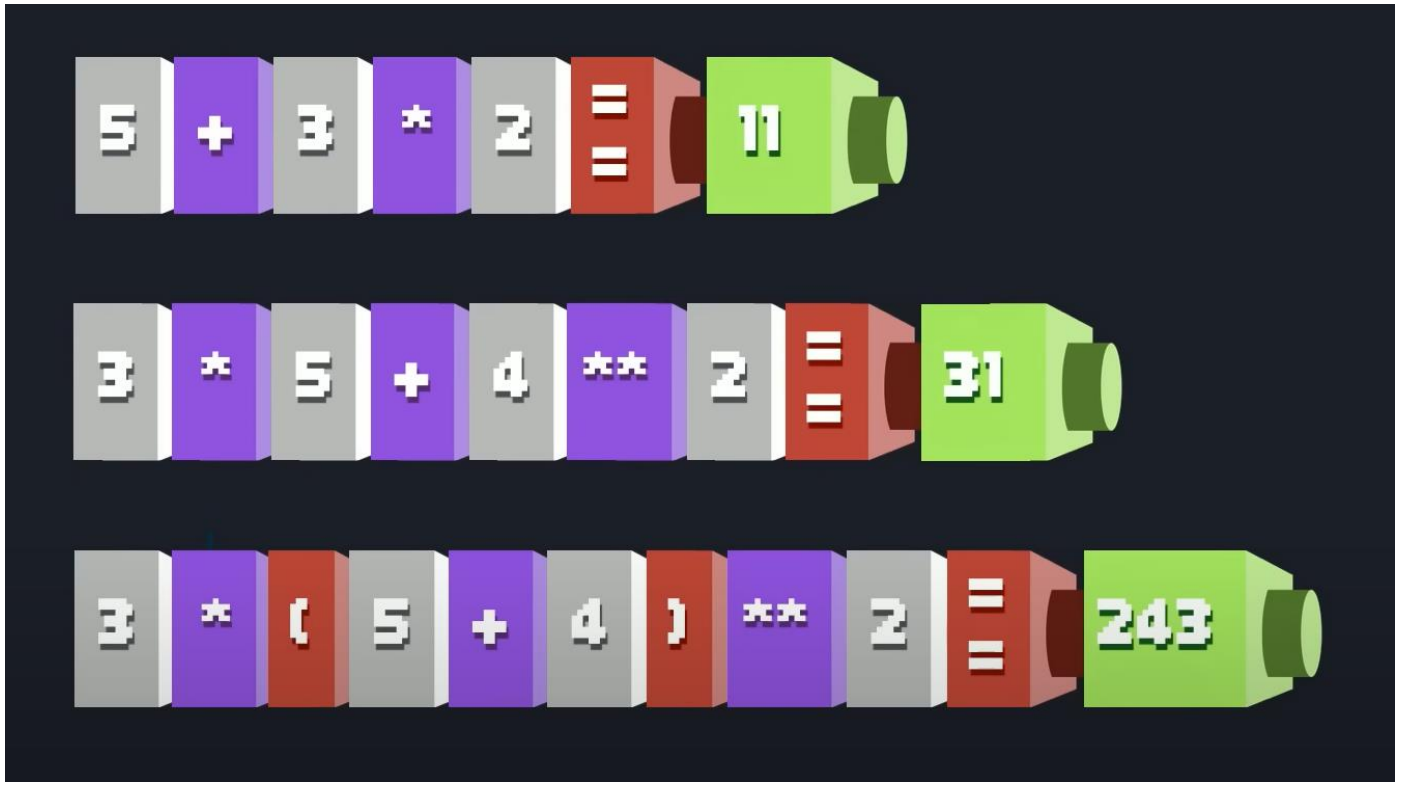
- Inteiro (int): Representa números inteiros, positivos ou negativos, sem parte decimal. Por exemplo: 42, -10, 0.
- Ponto flutuante (float): Representa números reais, ou seja, números com parte decimal. Por exemplo: 3.14, -0.001, 2.0.
- String (str): Representa texto. Strings são usadas para armazenar sequências de caracteres, como palavras ou frases. Por exemplo: 'Olá, mundo!', 'Python é incrível'.
- Booleano (bool): Representa um valor booleano, que pode ser **True** (verdadeiro) ou **False** (falso). Booleans são usados em expressões lógicas e de controle de fluxo. Por exemplo: True, False.
- Lista (list): Representa uma coleção ordenada de itens. Os itens em uma lista podem ser de tipos diferentes e podem ser alterados (mutáveis). Por exemplo: [1, 2, 3, 4], ['a', 'b', 'c'].

OPERADORES ARITMÉTICOS

OPERADORES ARITMÉTICOS



OPERADORES ARITMÉTICOS – ORDEM DE PRECEDÊNCIA



OPERADORES DE ATRIBUIÇÃO

Operador	Exemplo	Equivalente a
=	x = 1	x = 1
+=	x += 1	x = x + 1
-=	x -= 1	x = x - 1
*=	x *= 1	x = x * 1
/=	x /= 1	x = x / 1
%=	x %= 1	x = x % 1

OPERADORES DE COMPARAÇÃO ou OPERADORES RELACIONAIS

Operador	Conceito	Exemplo
>(Maior que)	Verifica se um valor é maior que outro	x > 5
<(Menor que)	Verifica se um valor é menor que outro	x < 5
== (Igual a)	Verifica se um valor é igual a outro	x == 5
!= (Diferente de)	Verifica se um valor é diferente de outro	x != 5
>= (Maior ou igual a)	Verifica se um valor é maior ou igual a outro	x >= 5
<= (Menor ou igual a)	Verifica se um valor é menor ou igual a outro	x <= 5

FIAP

EXERCÍCIOS

Python, Variáveis e Operadores

EXERCÍCIO 1

- Calcule a área de um círculo com raio 5. Use a fórmula: área = π * raio^2. (Dica: você pode usar 3.14159 como valor aproximado de π).

EXERCÍCIO 2

- Converta uma temperatura de Fahrenheit para Celsius. A fórmula de conversão é: Celsius = (Fahrenheit - 32) * 5/9.

EXERCÍCIO 3

- Converta uma temperatura de Fahrenheit para Celsius. A fórmula de conversão é: Celsius = (Fahrenheit - 32) * 5/9.

EXERCÍCIO 4

- Você comprou 3 livros por R$ 25 cada e 2 canetas por R$ 5 cada. Calcule o total gasto.

EXERCÍCIO 5

- Um carro percorreu 150 km a uma velocidade média de 60 km/h. Quanto tempo (em horas) o carro levou para percorrer essa distância?

EXERCÍCIO 6

- Leia 2 valores A e B, que correspondem a 2 notas de um aluno. A seguir calcule e informe a média aritmética do aluno.

EXERCÍCIO 7

- Leia 2 valores A e B, que correspondem a 2 notas de um aluno. A seguir calcule e informe a média ponderada do aluno, sabendo que a nota A tem peso 4 e a nota B tem peso 6.
- Exemplo: nota a * 4 e nota b * 6.

EXERCÍCIO 8

- Neste problema, deve-se ler o nome de uma peça que chamaremos de peça1, o número de peças1 que o usuário quer, o valor unitário de cada peça1, o nome de uma peça2, o número de peças2 e o valor unitário de cada peça2. Após, calcule e mostre o valor a ser pago.

EXERCÍCIO 9

- Crie um programa que receba o valor do produto e valor pago.
- Calcule o troco a ser pago.
- O valor do troco deve ser exibido no terminal.

ALGUMA DÚVIDA?

Entre em contato pelo Teams ou por e-mail

Alexandre Russi Junior

Projects Coordinator | Higher Education Professor

/alexandrerussi